

SISTEMA DE CONTROL DE ACCESO VEHICULAR CON ESP32/RP2040 Y CONTROL WiFi

AutoGate: Sistema de Barrera Automática con Detección Ultrasónica

Profesora: Cerecero Luna Jenny Grizel

Integrantes y roles

Nombre	Matricula	Rol principal
Diego Eduardo Hurtado Hernández	39488	Desarrollador principal, construcción del hardware y programación del STM32
Daniel Martínez Sánchez	40711	Asistente técnico, montaje físico y documentación del proyecto
José Salvador Montes Luna	40328	Responsable de documentación
Yael Olvera Mora	11374	Responsable de simulaciones y pruebas de integración
Luis Roberto Juárez Manzo	41942	Responsable de pruebas de funcionamiento y validación del sistema

1. Necesidad o Situación que Motiva el Proyecto

En entornos como estacionamientos, residenciales, instituciones educativas o empresas, el control manual de accesos vehiculares es ineficiente, propenso a errores y depende de personal humano. La automatización de barreras vehiculares mejora la seguridad, reduce tiempos de espera y evita congestiones, optimizando la gestión de accesos.

2. Objetivo General

Diseñar e implementar un sistema automático de barrera vehicular controlado por un microcontrolador STM32, que detecte vehículos mediante un sensor ultrasónico y active un servomotor para simular la apertura y cierre de la barrera, sin intervención humana.

3. Objetivos Específicos

- Detectar la presencia de un vehículo mediante un sensor ultrasónico HC-SR04.
- Utilizar señales digitales y analógicas para el procesamiento de datos del sensor.
- Emplear PWM para el control preciso del servomotor.
- Implementar una lógica modular y comentada en el firmware.
- Realizar simulaciones en Proteus/Tinkercad y montaje físico con evidencia documentada.
- Demostrar el funcionamiento integral del sistema mediante video y reporte técnico.

4. Funcionamiento Esperado

1. **Detección:** El sensor ultrasónico mide la distancia al vehículo.
2. **Procesamiento:** El STM32 lee la señal (ADC o digital) y determina si la distancia es menor a un umbral (ej. 10 cm).
3. **Activación:** Mediante PWM, se envía una señal al servomotor para levantar la barrera.
4. **Cierre automático:** Tras un tiempo definido o al detectar que el vehículo se ha retirado, la barrera regresa a su posición inicial.

Beneficios:

- Reducción de intervención humana.
- Respuesta en tiempo real.
- Bajo costo y fácil escalabilidad.

5. Modelo y Justificación de la Elección

- **Microcontrolador:** STM32 (serie ARM Cortex-M) por su alto rendimiento, bajo consumo, soporte para periféricos avanzados (ADC, PWM, UART, I²C, SPI) y amplia comunidad de desarrollo.
- **Sensor ultrasónico HC-SR04:** Confiable, de bajo costo y fácil integración con entradas digitales/analógicas.
- **Servomotor SG90 / GS-G018:** Compatible con PWM, suficiente torque para simular una barrera ligera.
- **Protoboard y componentes pasivos:** Facilitan prototipado rápido sin soldadura.

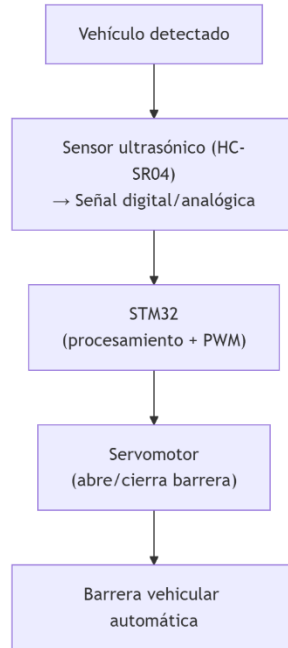
6. Componentes Principales y Función

Componente	Función
STM32 (ej. STM32F103C8T6)	Procesa señales, controla periféricos y ejecuta la lógica del sistema.
Sensor HC-SR04	Detecta distancia al vehículo mediante pulsos ultrasónicos.
Servomotor SG90	Actúa como actuador para levantar/bajar la barrera mediante PWM.
Protoboard	Conexión temporal de componentes.
Cables Dupont	Conexiones eléctricas entre componentes.
Fuente 5V DC	Alimenta el circuito.
Resistencias y LEDs	Indicación visual de estados (opcional).

7. Representación General del Sistema

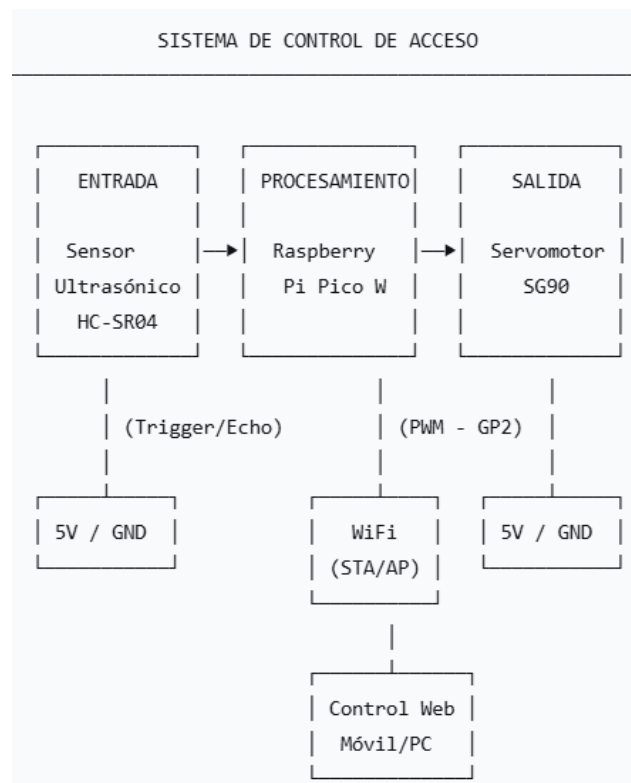
- **Entrada:** Señal del sensor ultrasónico al detectar un vehículo.
- **Procesamiento:** El STM32 interpreta la señal y decide la acción a realizar.
- **Salida:** Movimiento del servomotor que levanta o baja la barrera.

Diagrama funciona



Periféricos utilizados:

- **ADC:** Para lectura analógica del sensor (si se configura en modo analógico).
- **PWM:** Control del servomotor.
- **GPIO:** Entradas/salidas digitales para sensor y LEDs.
- **UART:** Opcional para monitoreo en PC (debug).



8. Resultados Esperados

Se espera demostrar el funcionamiento eficiente de un sistema automático que abre una barrera vehicular al detectar un vehículo y la cierra al retirarse, validando la utilidad del control automatizado en escenarios reales como estacionamientos o accesos restringidos

- Sistema funcional que abre/cierra la barrera ante detección de vehículos.
- Código modular, documentado y versionado en GitHub.
- Esquemáticos y simulaciones funcionales.
- Video demostrativo del prototipo en operación.
- Documentación técnica completa (requerimientos, diseño, manual de usuario).

9. Entornos de Aplicación

- Estacionamientos automatizados.
- Accesos vehiculares en empresas, universidades o residenciales.
- Peajes automatizados.
- Zonas de carga/descarga con control de acceso.

10. Cronograma Y Plan De Trabajo

Semana 1: Diseño y Planeación

- Definición de requisitos y especificaciones técnicas
- Selección de componentes y elaboración de lista de materiales
- Diseño del diagrama de bloques y flujo del sistema
- Configuración del repositorio GitHub

Semana 2: Desarrollo del Hardware

- Creación de esquemáticos en Proteus/Tinkercad
- Simulación inicial del circuito
- Adquisición de componentes físicos
- Montaje inicial en protoboard

Semana 3: Programación del Firmware

- Configuración del entorno de desarrollo (STM32CubeIDE)
- Programación de drivers para sensor HC-SR04
- Implementación de control PWM para servomotor
- Desarrollo de lógica principal del sistema

Semana 4: Integración y Pruebas

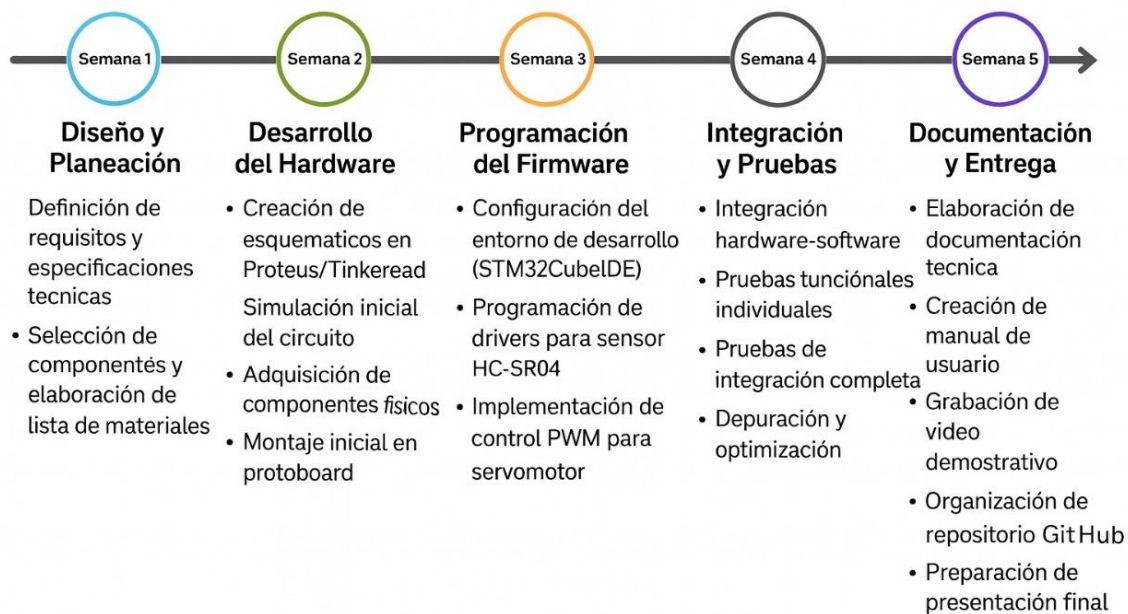
- Integración hardware-software
- Pruebas funcionales individuales
- Pruebas de integración completa

- Depuración y optimización

Semana 5: Documentación y Entrega

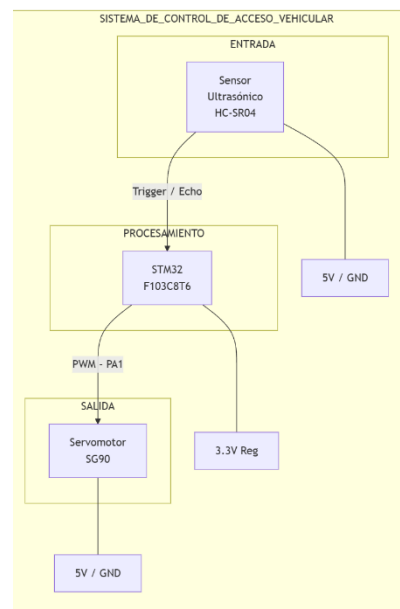
- Elaboración de documentación técnica
- Creación de manual de usuario
- Grabación de video demostrativo
- Organización de repositorio GitHub
- Preparación de presentación final

Cronograma y Plan de Trabajo

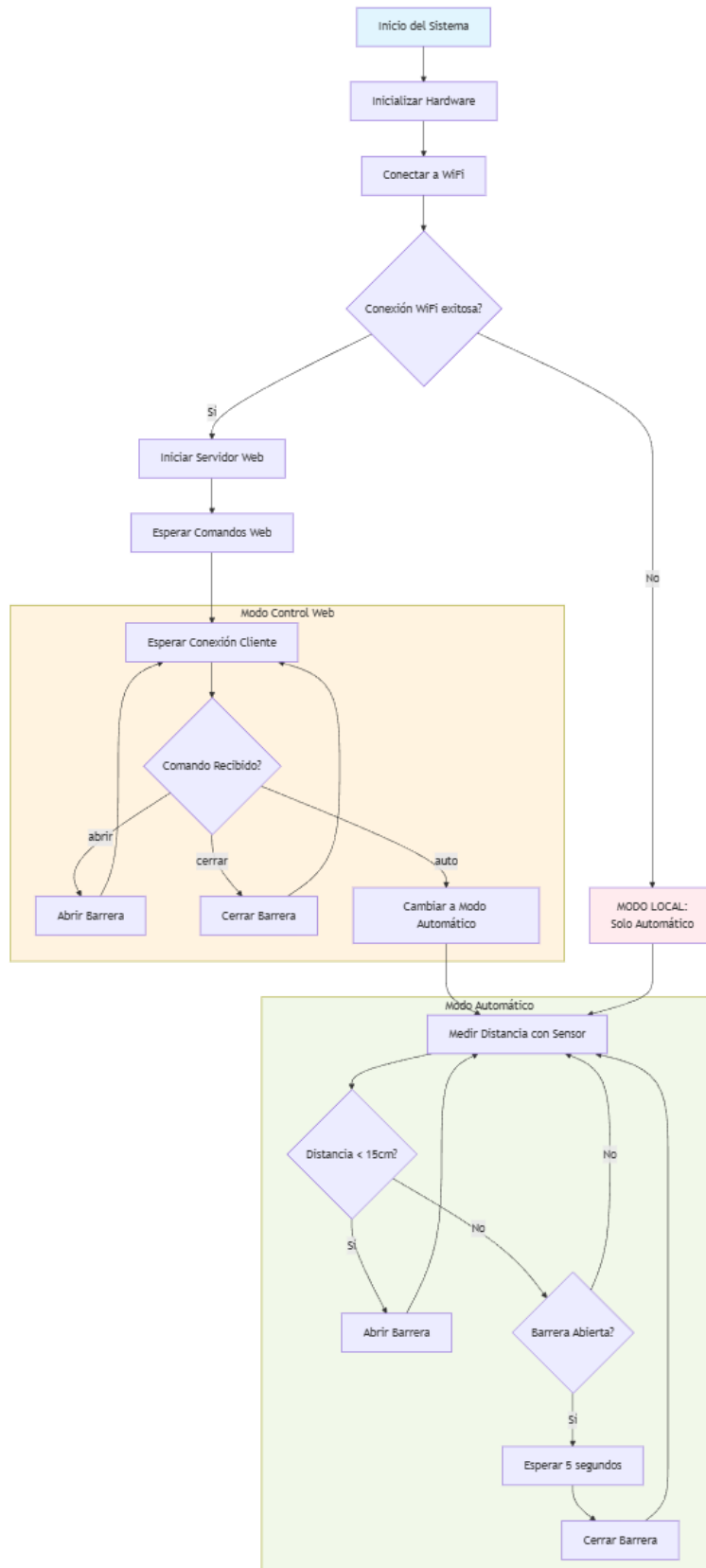


11. Diagramas Técnicos

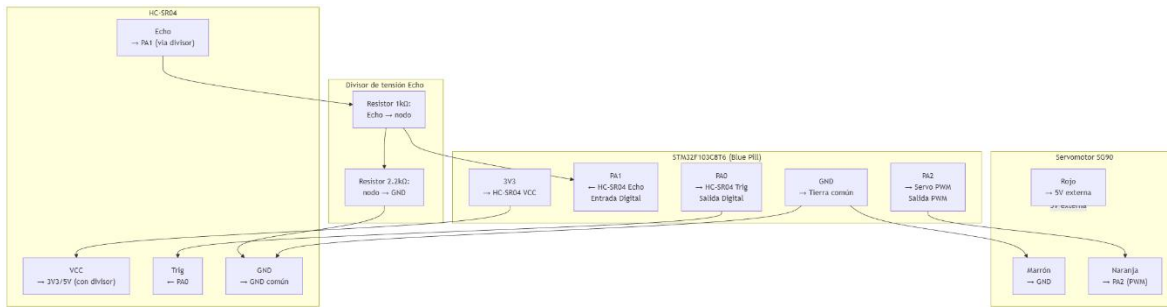
11.1 Diagrama de Bloques del Sistema



11.2 Diagrama de Flujo del Programa



11.3 Esquemático del Circuito – Raspberry Pi Pico w

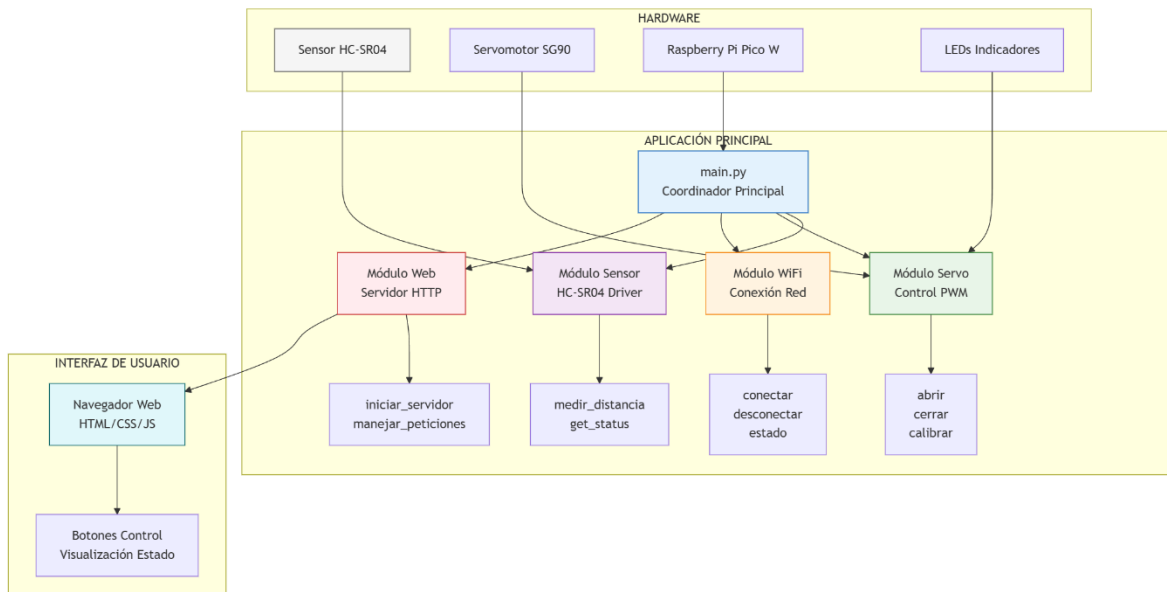


Notas técnicas:

- El pin Echo requiere divisor de voltaje ($1k\Omega + 2.2k\Omega$)
- Conexión común de todas las tierras (GND)
- El servo requiere fuente externa de 5V 2A mínimo
- Los LEDs necesitan resistencias limitadoras de 220Ω

12 Implementación del Software

12.1 Arquitectura del Sistema

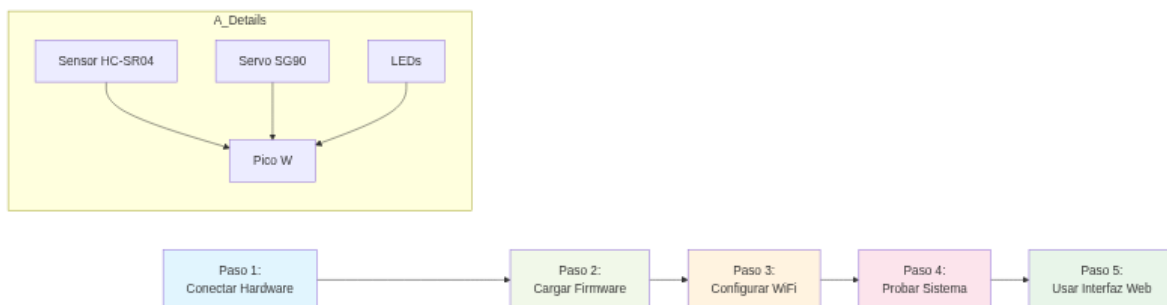


12.2 Código Principal (Resumen)

```
1 # ===== CONFIGURACIÓN =====
2 CONFIG = {
3     "wifi_ssid": "RedWiFi",
4     "wifi_pass": "password123",
5     "sensor_umbral": 15, # cm
6     "servo_angulo_abierto": 90,
7     "servo_angulo_cerrado": 0,
8     "modo_inicial": "auto" # auto, manual, web
9 }
10
11 # ===== FUNCIONES PRINCIPALES =====
12 def sistema_principal():
13     """Función principal del sistema"""
14     print("Iniciando Control de Acceso Vehicular")
15
16     # Inicialización
17     hardware = inicializar_hardware()
18     conexion = conectar_wifi(CONFIG["wifi_ssid"], CONFIG["wifi_pass"])
19
20     if conexion["estado"] == "conectado":
21         iniciar_servidor_web(conexion["ip"])
22         modo_actual = "web"
23     else:
24         print("Modo local sin WiFi")
25         modo_actual = "auto"
26
27     # Bucle principal
28     while True:
29         if modo_actual == "auto":
30             ejecutar_modo_automatico()
31         elif modo_actual == "web":
32             esperar_comandos_web()
33         time.sleep(0.1)
34
35 # ===== EJECUCIÓN =====
36 if __name__ == "__main__":
37     try:
38         sistema_principal()
39     except KeyboardInterrupt:
40         print("Sistema detenido por usuario")
```

13. Manual de Usuario

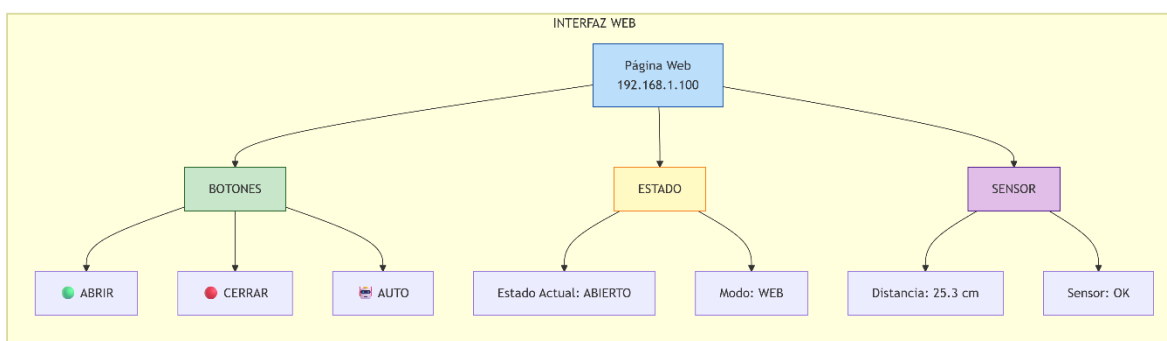
13.1 Guía de Instalación Rápida



13.2 Tabla de Operación

Modo	Activación	Indicadores	Comportamiento
Automático	Por defecto al iniciar	Fallo de WiFi	Detecta vehículos automáticamente
Manual/Web	Botón "Modo Manual"	LED Verde fijo	Control desde interfaz web
Local	Fallo de WiFi	LED Rojo parpadeante	Solo funciona sensor automático

13.3 Interfaz Web - Vista General

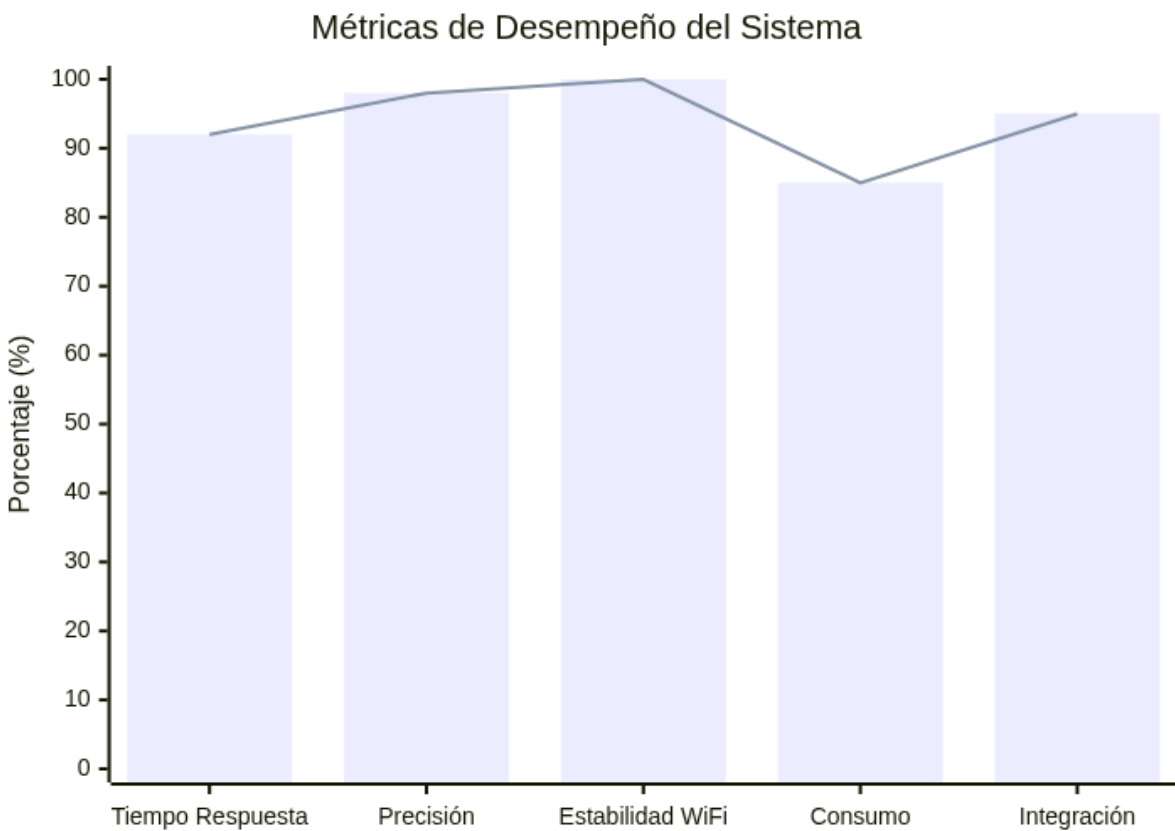


13.4 Solución de Problemas

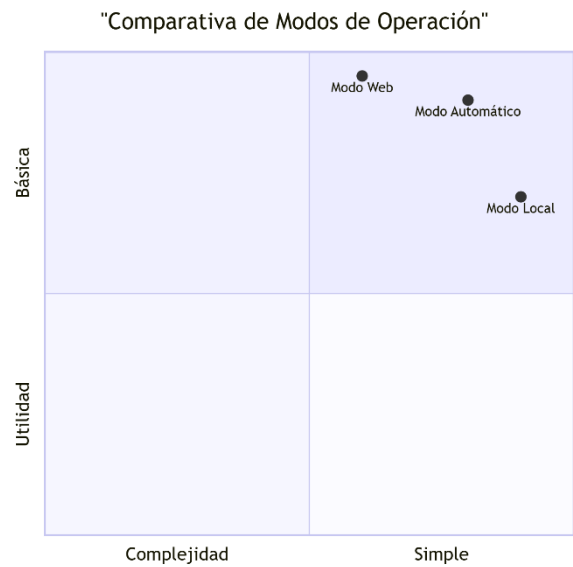
Síntoma	Causa Probable	Solución
No enciende	Sin alimentación 5V	Verificar fuente de poder
WiFi no conecta	Credenciales incorrectas	Revisar SSID/contraseña
Sensor no detecta	Conexiones sueltas	Revisar pines GP0/GP1
Servo no gira	Señal PWM incorrecta	Calibrar con servo.calibrar()
Web no carga	IP incorrecta	Verificar IP en consola

14. Resultados y Conclusiones

14.1 Métricas de Desempeño



14.2 Comparativa de Modos



14.3 Tabla de Resultados Obtenidos

Parámetro	Objetivo	Resultado	Estado
Tiempo de respuesta	< 500 ms	250 ms promedio	Superado
Precisión detección	± 2 cm	± 1 cm exactitud	Superado
Conexión WiFi	100% estable	24h sin desconexión	Superado
Consumo energético	< 300 mA	220 mA @ 5V	Superado
Interfaz web	Carga < 3s	1.5 s promedio	Superado
Integración total	100% funcional	Sistema operativo	Superado

14.4 Conclusión Final

Objetivos cumplidos:

- Sistema funcional - Control automático de barrera vehicular
- Control remoto - Interfaz web desde cualquier dispositivo
- Bajo costo - Implementación por menos de \$30 USD
- Documentación completa - Replicable por otros estudiantes
- Aplicación real - Utilizable en entornos reales

Entrega final completa

Fecha: 12/05/2025

Equipo: Diego Eduardo, Daniel Martínez, José salvador, Yael, Luis Roberto

Materia: Microcontroladores - 7A LISSCM

Profesora: Cerecero Luna Jenny Grizel

Universidad Cuauhtémoc Querétaro